Matière : **Conception et développement d'applications**
Thème : **Préparation épreuve E6 BTS SIO option SLAM**
Application 1 : **Cahier des charges techniques**

Projet : Site web des Jeux Olympiques 2028

Réalisé par : Evan MICHINEAU

Sommaire

1. Contexte du projet....2
1.1. Présentation du projet....2
1.2. Date de rendu du projet....2
2. Besoins fonctionnels....2
3. Ressources nécessaires à la réalisation du projet....2
3.1. Ressources matérielles....2
3.2. Ressources logicielles....2
4. Gestion du projet....3
5. Conception du projet....3
5.1. Le front-end....3
5.1.1. Wireframes....3
5.1.2. Maquettes....5
5.1.3. Arborescences....7
5.2. Le back-end....7
5.2.1. Diagramme de cas d'utilisation....7
5.2.2. Diagramme d'activités....8
5.2.3. Modèle Conceptuel de Données (MCD)....8
5.2.4. Modèle Logique de Données (MLD)....9
5.2.5. Modèle Physique de Données (MPD)....9
6. Technologies utilisées....10
6.1. Langages de développement Web....10
6.2. Base de données....10
7. Sécurité....10
7.1. Login et protection des pages administrateurs....10
7.2. Hachage des mots de passe avec Bcrypt....10
7.3. Protection contre les attaques XSS (Cross-Site Scripting)....10
7.4. Protection contre les injections SQL....11
8. Tests....11
8.1. Tests unitaires avec PHPUnit....11

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le [date à compléter].

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

La partie publique (accessible à tous, sans compte) doit permettre de :

- consulter la liste des sports ;
- consulter le calendrier des épreuves (sport, date, heure, lieu) ;
- consulter les résultats (sport, athlète, pays, résultat) ;
- accéder à la page de connexion administrateur.

La partie privée (réservée aux administrateurs connectés) doit permettre de gérer, avec l'ajout, la modification et la suppression des données :

- les administrateurs ;
- les sports ;
- les lieux ;
- les pays ;
- les genres ;
- les athlètes ;
- le calendrier des épreuves ;
- les résultats.

Il ne doit pas être possible d'ajouter deux fois la même donnée, et on ne peut pas supprimer une donnée qui est encore utilisée ailleurs (par exemple un sport qui a des épreuves).

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

Pour développer le site, nous avons besoin de :

- un ordinateur (8 Go de RAM minimum) ;
- une connexion Internet ;
- un smartphone ou une tablette pour tester l'affichage responsive ;
- une clé USB ou un disque externe pour sauvegarder le projet ;
- un serveur d'hébergement distant (fourni par InfinityFree) pour la mise en ligne.

3.2. Ressources logicielles

Logiciels et plateformes utilisés :

- MAMP ou WAMP : serveur local (Apache, PHP, MySQL) ;
- phpMyAdmin : gestion et import de la base de données ;
- Visual Studio Code : éditeur de code ;
- Chrome ou Firefox : tests et outils de développement ;
- Composer et PHPUnit : tests unitaires ;

– Figma ou Mockflow : wireframes ;
– Visual Paradigm et Mocodo : diagrammes UML et modèles de données ;
– Git et GitHub : gestion de versions ;
– FileZilla et InfinityFree : mise en ligne ;
– Word ou LibreOffice : rédaction des documents ;
– Cyclades : dépôt de la fiche descriptive.

Sources documentaires : php.net, MDN Web Docs, documentation MySQL, OWASP, CNIL, documentation de PHPUnit, de Git/GitHub et d'InfinityFree.

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.

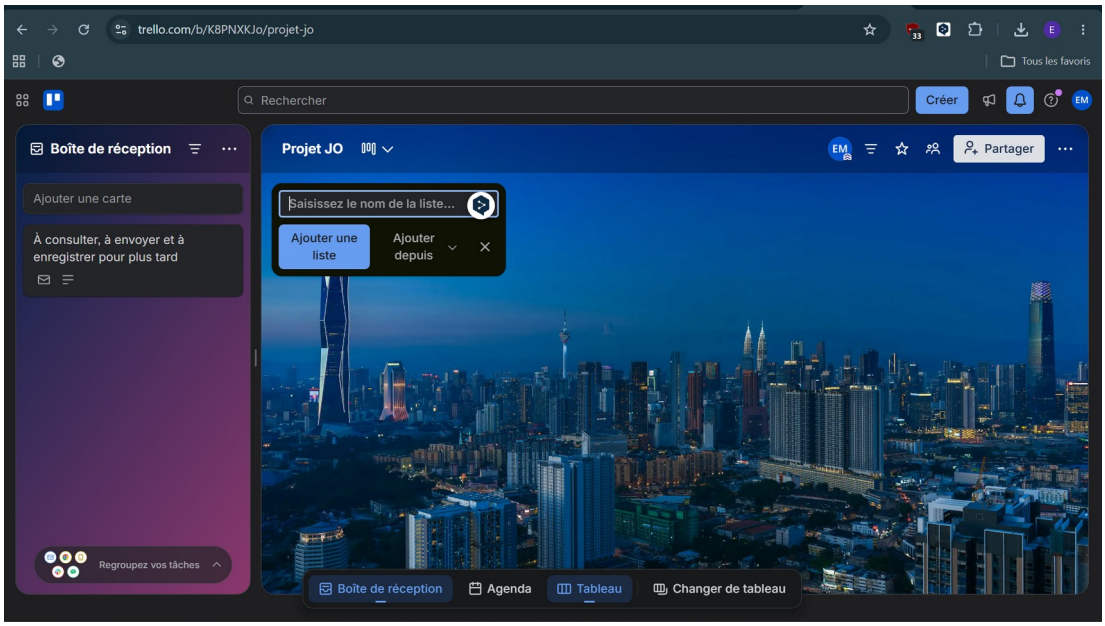


Tableau Trello du projet JO

Nous travaillons également sur GitHub, plateforme de développement collaboratif. Le dépôt du projet s'appelle jo2028-michineau-evan.

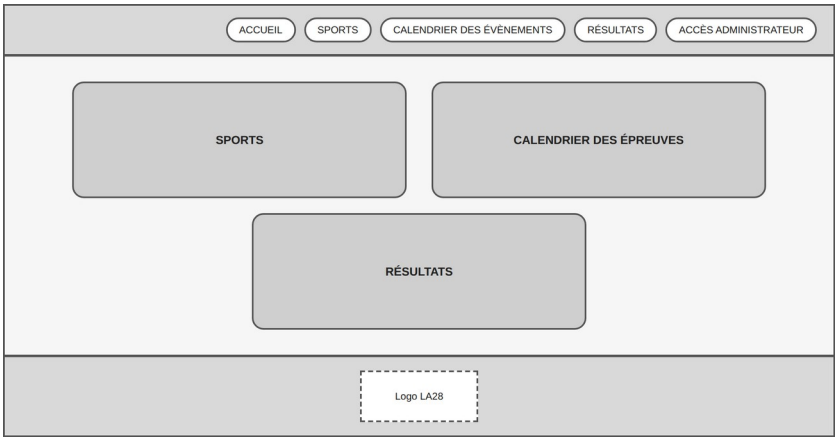
5. Conception du projet

5.1. Le front-end

5.1.1. Wireframes

Les wireframes ont été réalisés avec Figma. Ils montrent la structure des pages principales du site (accueil, listes, connexion), sans couleurs ni images, en version ordinateur puis en version smartphone/tablette.

Lien du fichier Figma : https://www.figma.com/design/mTAxzEXHRqJ5Xjsr4OIFQn/Wireframes-JO-2028



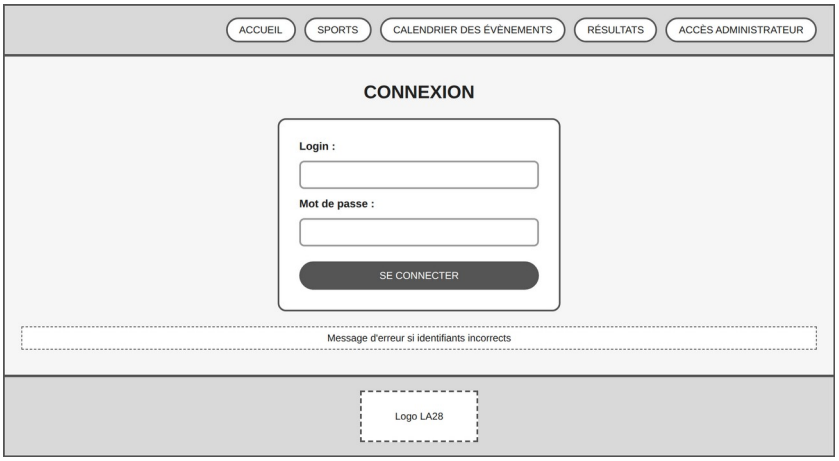
Wireframe de l'accueil (ordinateur)



Wireframe de la liste des résultats (ordinateur)

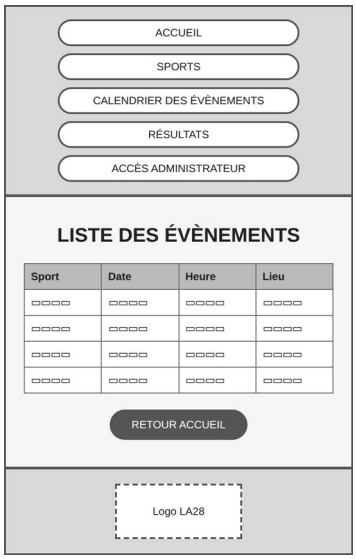


Wireframe de la liste du calendrier des évènements (ordinateur)



Wireframe de la page de connexion (ordinateur)

Version smartphone et tablette : le menu s'affiche en boutons empilés.



Calendrier



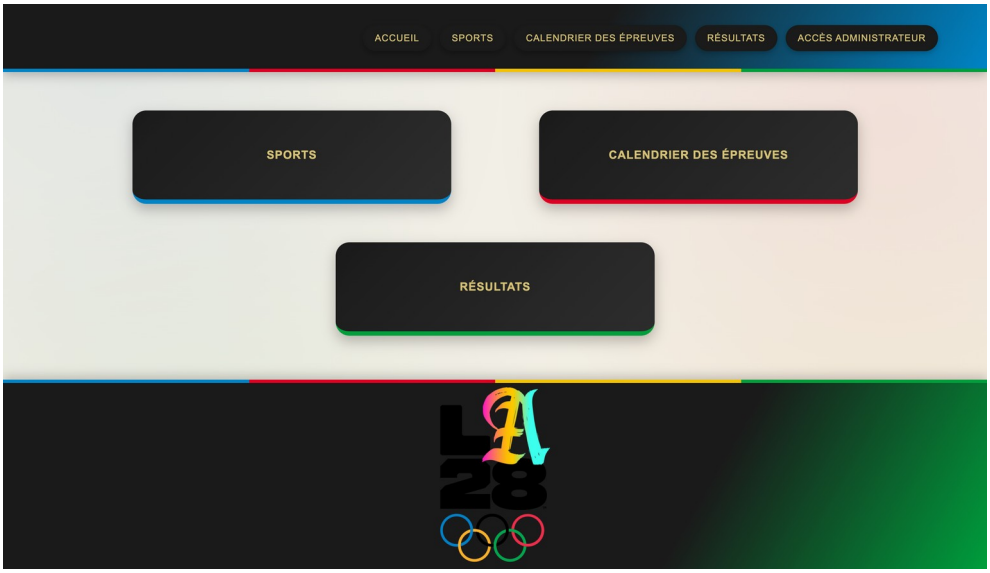
Liste des sports



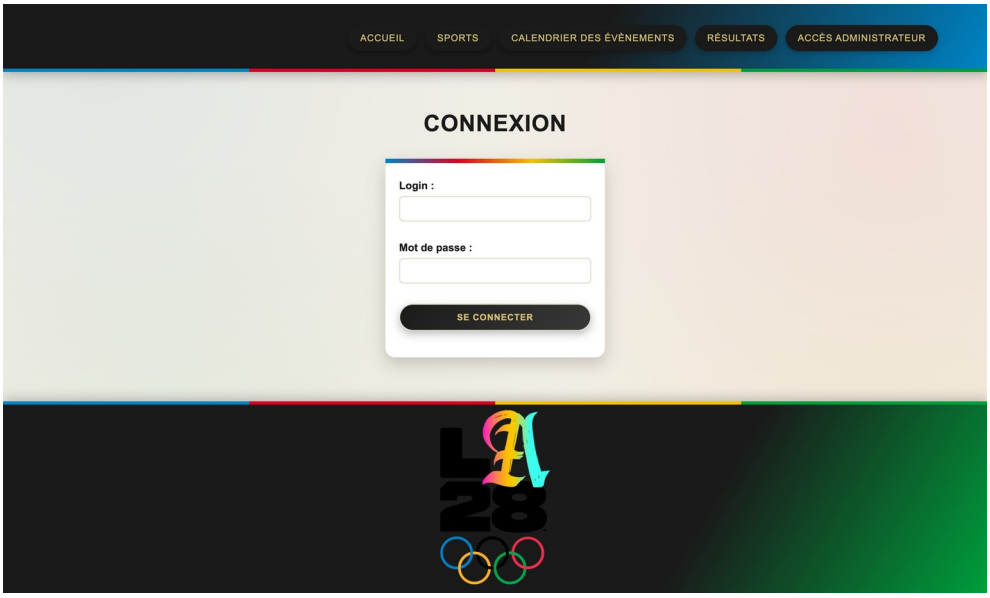
Connexion

5.1.2. Maquettes

Les maquettes ont été fournies par le client. Elles existent en version ordinateur, tablette et smartphone. Voici les principales :



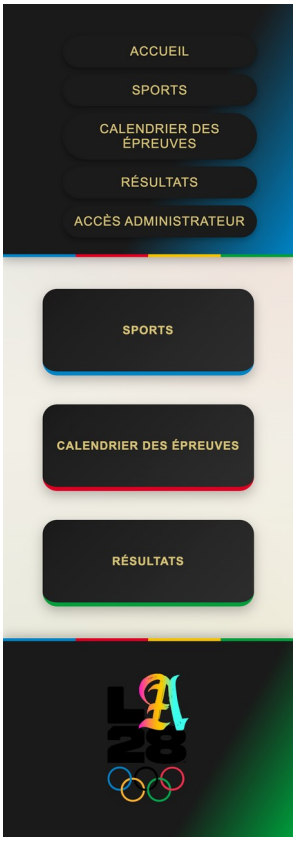
Maquette de la page d'accueil (ordinateur)



Maquette de la page de connexion (ordinateur)



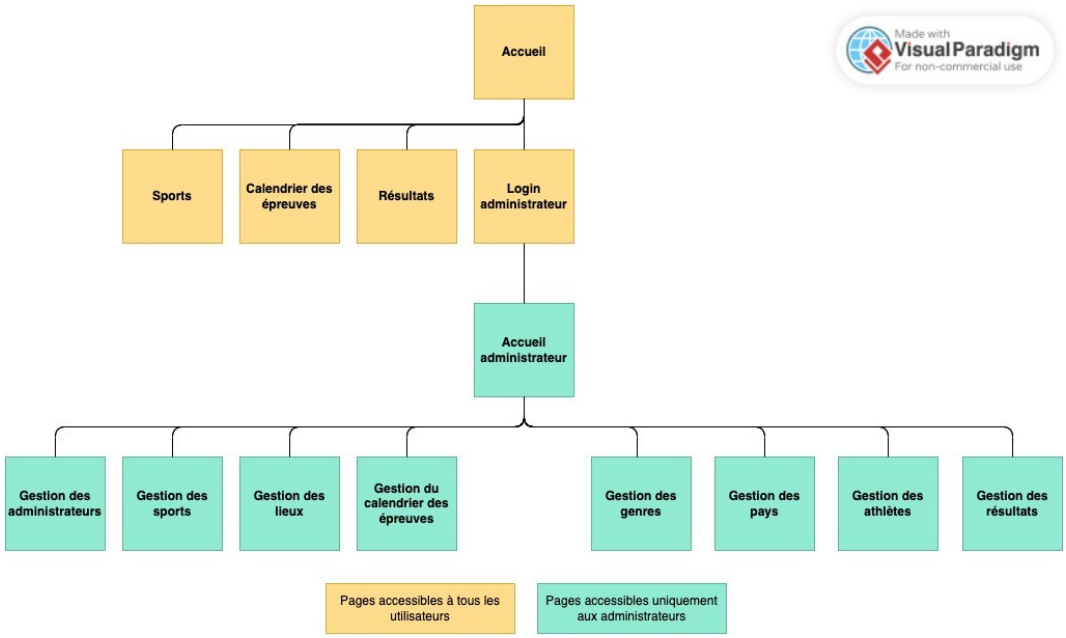
Maquette d'une page en tableau (ordinateur)



Maquette de l'accueil en version responsive (tablette et smartphone) : le menu s'affiche en boutons empilés

5.1.3. Arborescences

Le site compte 4 pages publiques (accueil, sports, calendrier des épreuves, résultats) et une page de connexion. Une fois connecté, l'administrateur arrive sur un accueil qui donne accès aux 8 pages de gestion.



Arborescence du site (jaune : pages publiques, vert : pages administrateur)

5.2. Le back-end

5.2.1. Diagramme de cas d'utilisation

Deux acteurs : l'usager consulte les informations, l'administrateur se connecte puis gère toutes les données.

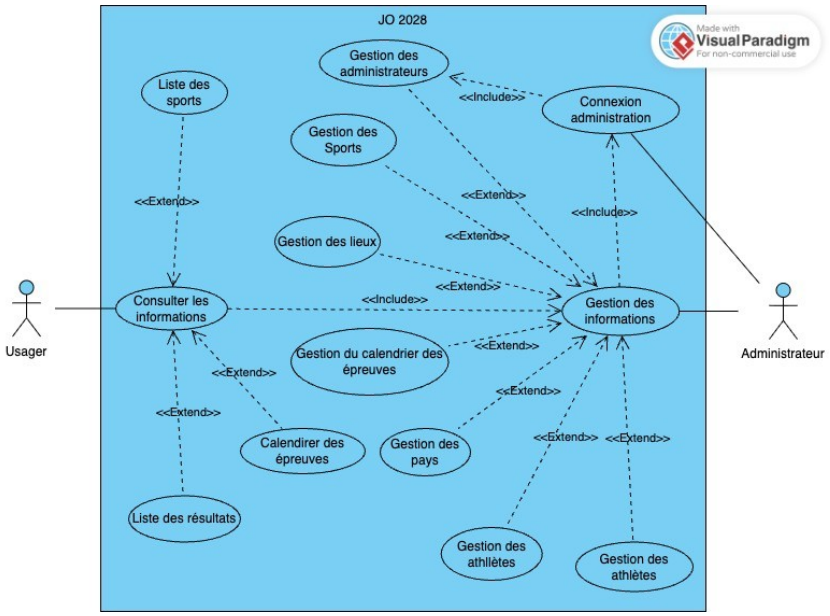


Diagramme de cas d'utilisation

5.2.2. Diagramme d'activités

Ce diagramme décrit la connexion d'un administrateur : saisie du login et du mot de passe, vérification, puis accès à l'espace administrateur ou message d'erreur.

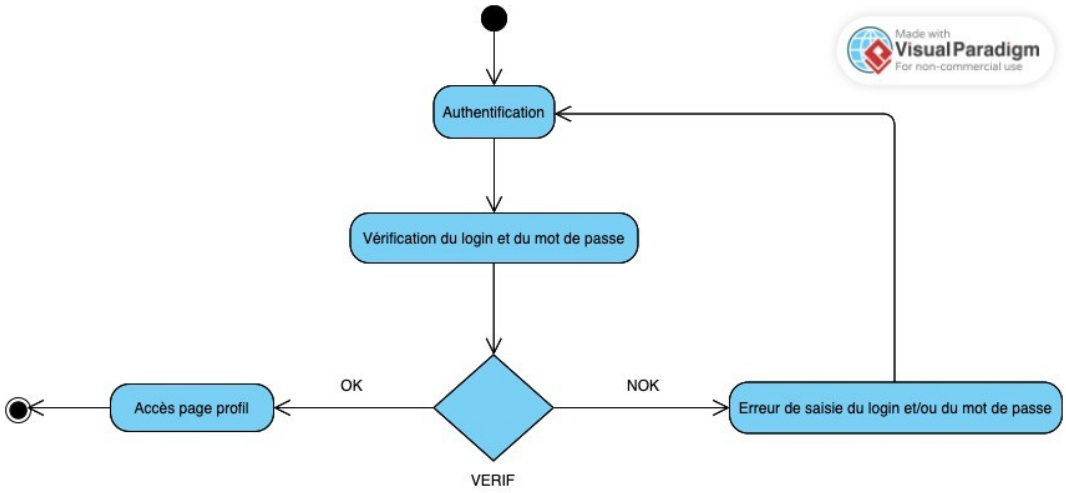
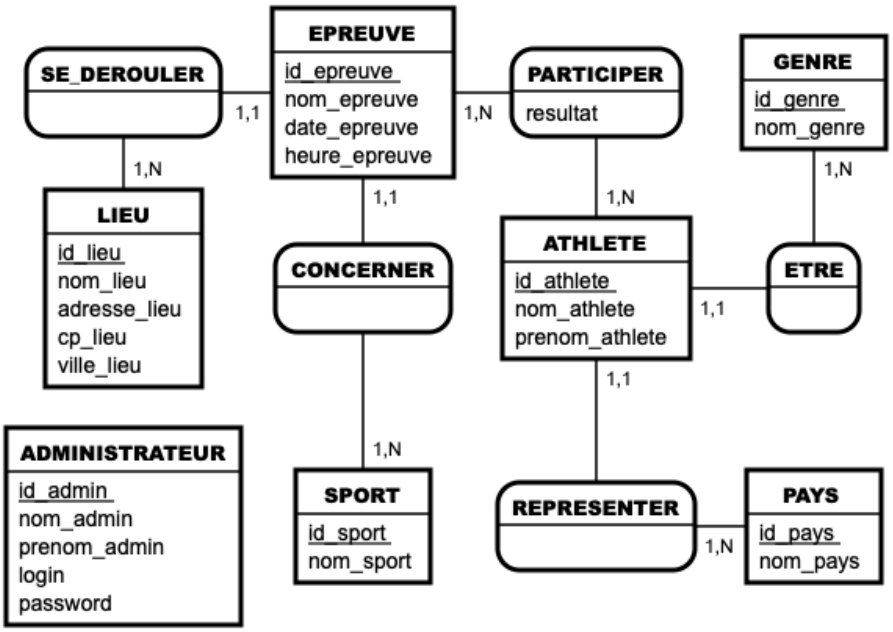


Diagramme d'activités de la connexion

5.2.3. Modèle Conceptuel de Données (MCD)

Le MCD montre les entités du projet et leurs liens : un athlète représente un pays et a un genre, une épreuve concerne un sport et a lieu dans un lieu, et les athlètes participent aux épreuves.



Modèle conceptuel de données (MCD)

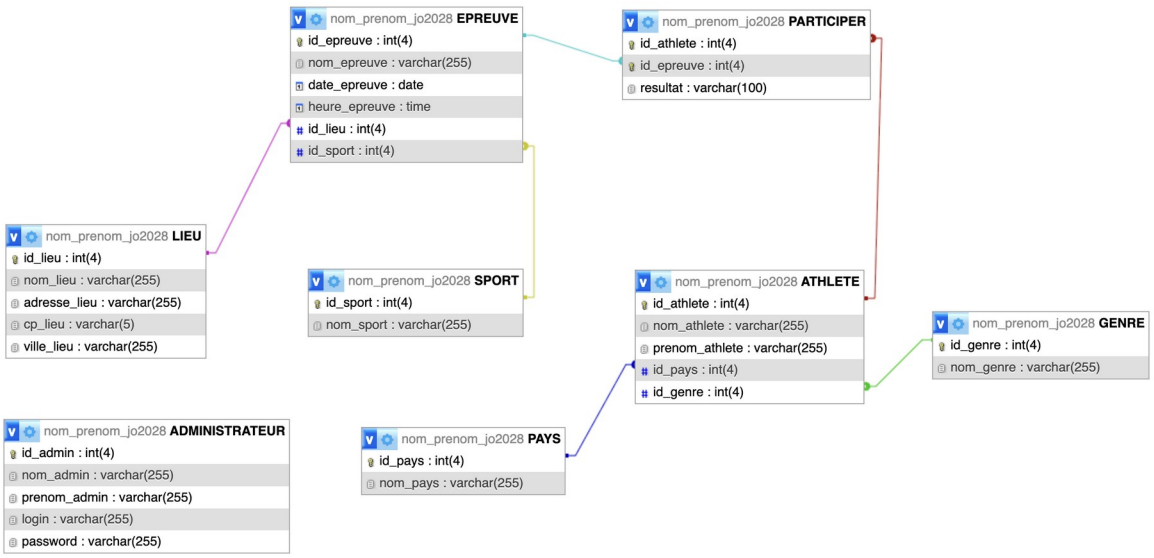
5.2.4. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables. Les clés primaires sont soulignées dans le modèle d'origine ; ici elles sont en premier et les clés étrangères sont précédées de #.

```
SPORT (id_sport, nom_sport)
PAYS (id_pays, nom_pays)
GENRE (id_genre, nom_genre)
LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)
ATHLETE (id_athlete, nom_athlete, prenom_athlete, #id_pays, #id_genre)
EPREUVE (id_epreuve, date_epreuve, heure_epreuve, #id_lieu, #id_sport)
PARTICIPER (#id_athlete, #id_epreuve, resultat)
ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)
```

5.2.5. Modèle Physique de Données (MPD)

Le MPD correspond aux tables telles qu'elles sont créées dans MySQL (types, clés primaires et étrangères).



Modèle physique de données (MPD)

6. Technologies utilisées

6.1. Langages de développement Web

Le site est développé avec :

- HTML5 : structure des pages ;
- CSS3 : mise en forme et affichage responsive (normalize.css, styles-computer.css, styles-responsive.css) ;
- PHP avec l'extension PDO : traitement côté serveur et connexion sécurisée à la base ;
- JavaScript : uniquement côté front-end (par exemple les confirmations de suppression) ;
- SQL : création des tables et requêtes.

6.2. Base de données

Les données sont enregistrées dans une base de données relationnelle MySQL, nommée michineau_evan_jo2028. Elle contient 8 tables (SPORT, PAYS, GENRE, LIEU, ATHLETE, EPREUVE, PARTICIPER, ADMINISTRATEUR). On la crée avec phpMyAdmin en important le fichier db-jo.sql (structure) puis test-data-jo.sql (données de test). La connexion se fait dans un seul fichier, database.php, avec PDO.

7. Sécurité

7.1. Login et protection des pages administrateurs

L'administrateur se connecte avec un login et un mot de passe. Le fichier auth.php cherche le login dans la table ADMINISTRATEUR, vérifie le mot de passe, puis enregistre ses informations en session :

```
if ($row && password_verify($password, $row["password"])) {
    session_regenerate_id(true);
    $_SESSION["id_admin"] = $row["id_admin"];
    $_SESSION["login"] = $row["login"];
    header("location: ../pages/admin/admin.php");
}
```

Toutes les pages d'administration commencent par inclure le fichier includes/init.php, qui démarre la session et vérifie que l'administrateur est connecté. S'il ne l'est pas, il est renvoyé vers l'accueil :

```
session_start();
if (!isset($_SESSION['login'])) {
    header('Location: ../../../index.php');
    exit();
}
```

À la déconnexion, la session est détruite. Les formulaires contiennent aussi un jeton CSRF, généré avec random_bytes() et vérifié à chaque envoi, pour empêcher qu'un site extérieur envoie une requête à la place de l'administrateur.

7.2. Hachage des mots de passe avec Bcrypt

Bcrypt est un algorithme de hachage conçu pour les mots de passe. Il transforme le mot de passe en une chaîne illisible, impossible à retransformer en mot de passe, et il est volontairement lent pour décourager les attaques par force brute. Ainsi, même si la base de données est volée, les mots de passe ne sont pas lisibles.

En PHP, on hache le mot de passe avant de l'enregistrer, puis on le compare à la connexion :

```
// Enregistrement d'un administrateur
$hash = password_hash($password, PASSWORD_BCRYPT);

// Connexion
password_verify($password, $hash);   // renvoie true ou false
```

Dans la base, la colonne password ne contient donc jamais le mot de passe en clair.

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

Une attaque XSS consiste à écrire du code (souvent du JavaScript) dans un champ de formulaire. Si le site affiche ce texte tel quel, le code s'exécute dans le navigateur des autres visiteurs, par exemple pour voler leur session.

Pour s'en protéger, nous retirons les espaces inutiles à la saisie et nous échappons toutes les données à l'affichage avec htmlspecialchars(), qui transforme les caractères comme < et > en texte inoffensif. Cette opération est regroupée dans une petite fonction e(), utilisée dans toutes les pages d'administration :

```
// includes/functions.php
function e($texte)
{
    return htmlspecialchars((string) $texte, ENT_QUOTES, 'UTF-8');
}

// Dans une page
echo e($ligne['nom_sport']);
```

7.4. Protection contre les injections SQL

Une injection SQL consiste à écrire du code SQL dans un champ de formulaire pour modifier la requête envoyée à la base (par exemple pour se connecter sans mot de passe ou supprimer des données).

Pour l'éviter, nous utilisons uniquement des requêtes préparées avec PDO. La valeur saisie est envoyée séparément de la requête, elle est donc toujours traitée comme une simple donnée et jamais comme du code :

```
$check = $connexion->prepare("SELECT COUNT(*) FROM SPORT WHERE nom_sport = :nom");
$check->bindParam(":nom", $nom, PDO::PARAM_STR);
$check->execute();
```

8. Tests

8.1. Tests unitaires avec PHPUnit

Pour vérifier que l'application fonctionne, nous utilisons PHPUnit, l'outil de tests unitaires de PHP, installé avec Composer. Les tests se trouvent dans le dossier tests/ du projet.

Ils utilisent une base de données de test séparée (michineau_evan_jo2028_test) pour ne jamais modifier les vraies données du site. Chaque test supprime les données qu'il a créées.

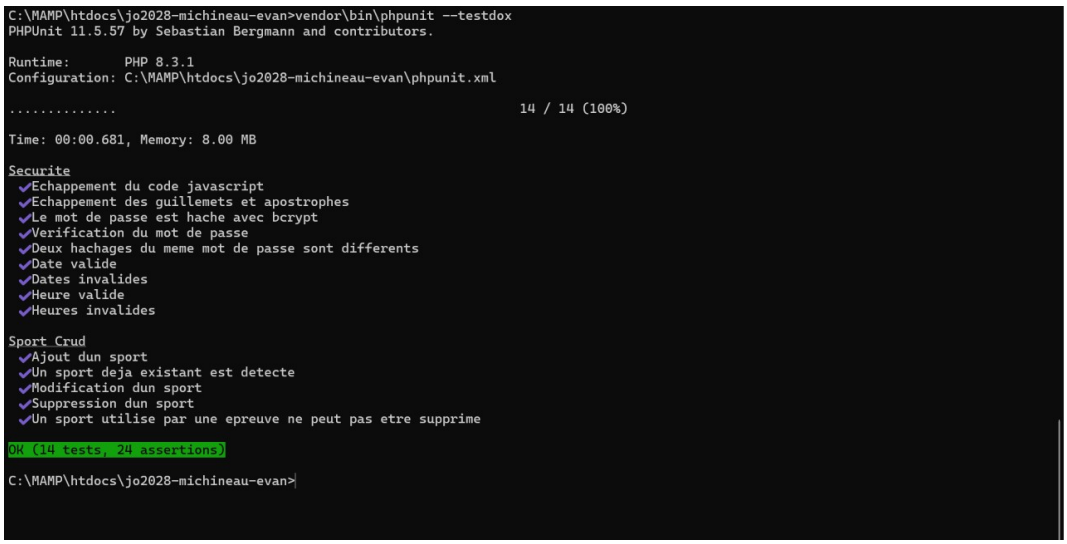
14 tests couvrent trois fonctionnalités clés :

- la sécurité : échappement du code JavaScript (XSS) et hachage Bcrypt des mots de passe (hachage commençant par $2y$, vérification du bon et du mauvais mot de passe) ;
- la validation des dates et des heures du calendrier des épreuves ;
- la gestion des sports : ajout, détection d'un doublon, modification, suppression, et refus de supprimer un sport utilisé par une épreuve.

Les tests se lancent avec la commande suivante :

```
vendor\bin\phpunit --testdox
```

Résultat obtenu : tous les tests réussissent (OK : 14 tests, 24 assertions).



Résultat de l'exécution des tests PHPUnit